

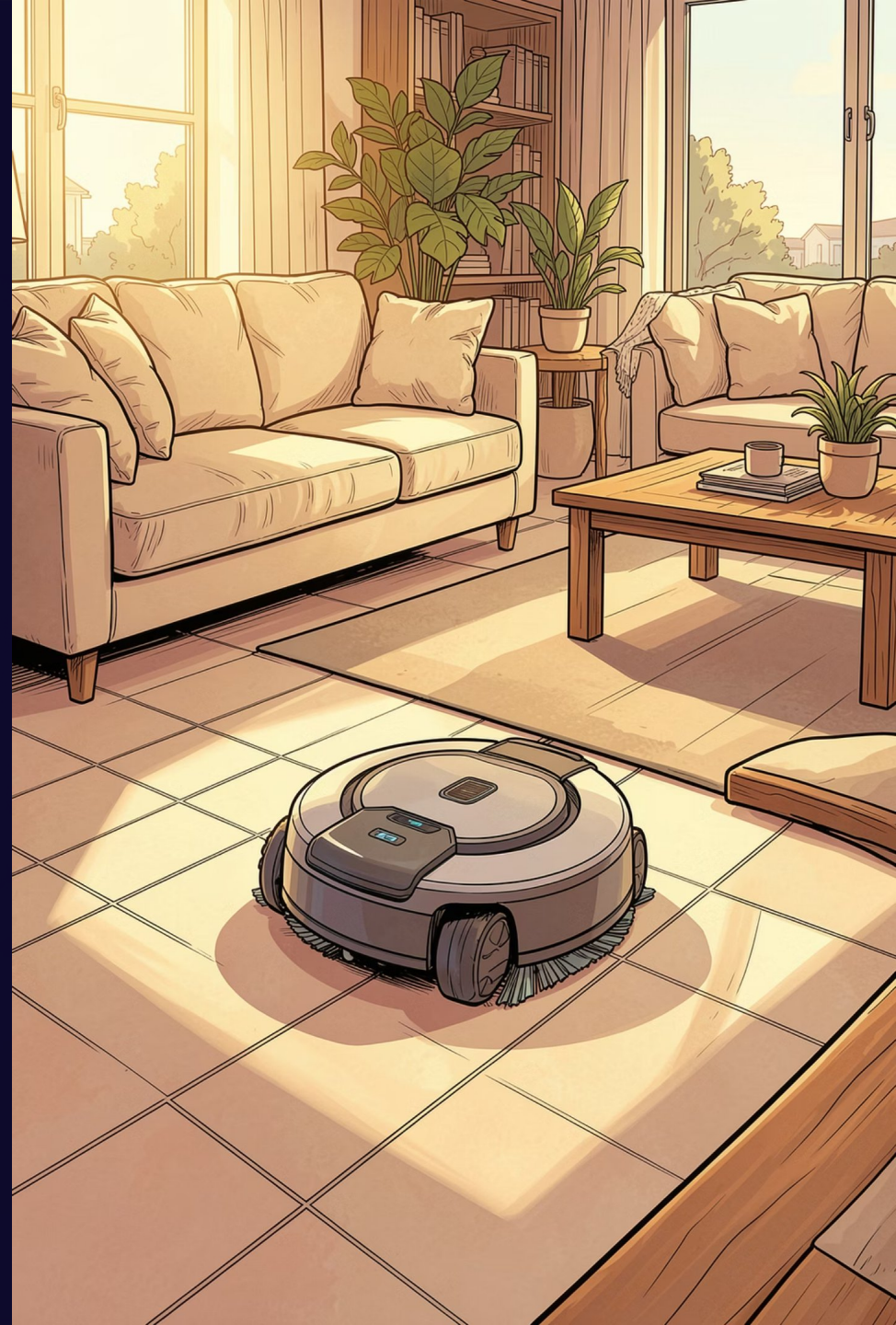
# Autonomous Cleaning Strategy Planner

Team Members:

2520090007- Dikshita Siripireddy

2520090062- Yerra Sharathchandra Goud

2520090082- Siddoju Anjani Shreyaj



# ABSTRACT

The Autonomous Cleaning Strategy Planner is an intelligent software system designed to generate efficient cleaning paths for indoor environments such as homes, offices, hospitals, and commercial spaces. The main objective of the project is to create an optimized cleaning strategy that ensures maximum area coverage in minimum time while avoiding obstacles and reducing unnecessary movements.

Traditional cleaning planning methods may lead to repeated coverage, missed areas, and inefficient navigation. To overcome these issues, the proposed system uses Artificial Intelligence techniques such as grid-based maps, path planning, heuristic search, and obstacle-aware navigation to determine the best possible cleaning route. The system analyses the floor layout, including rooms, walls, and obstacles, and produces a systematic cleaning plan for efficient area coverage.



# CO1 — PEAS Model

## Performance

Maximize percent coverage of accessible cells while minimizing energy and time.

## Environment

Discrete indoor grid (cells: free, obstacle, dirty). Static furniture, dynamic small obstacles possible.

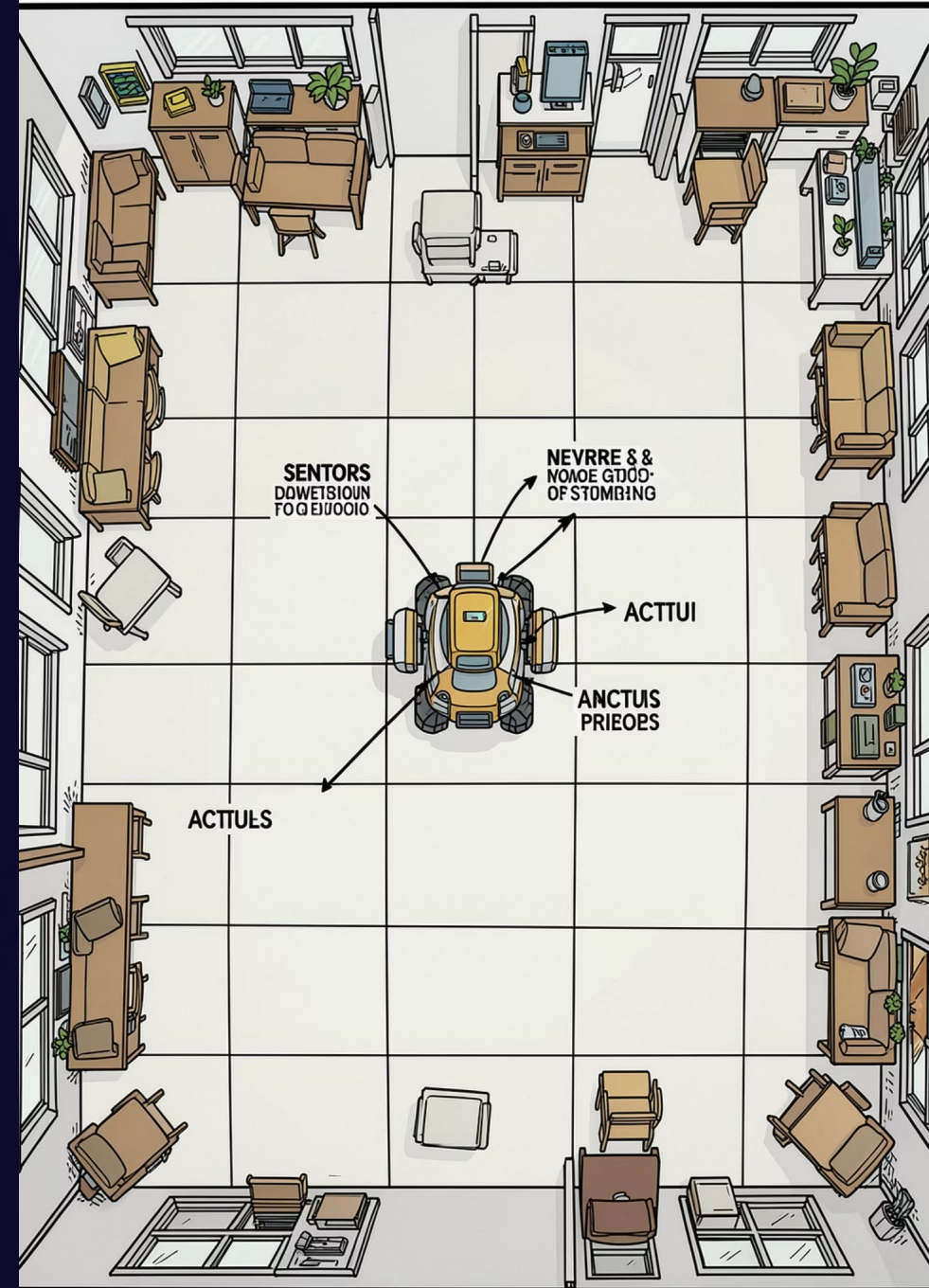
## Actuators

Move (N, S, E, W), rotate, start/stop vacuum, dock for charging.

## Sensors

Proximity/obstacle detector, bump sensor, dirt detector, localization (odometry + occasional landmarks).

**i** Example code snippet for logging: `print('Performance: coverage, energy, runtime')`



```

# ----- CO1 -----
# PEAS MODEL
print("\n PEAS MODEL")
print("Performance : Maximum cleaning coverage")
print("Environment : Indoor cleaning grid")
print("Actuators : Robot movement")
print("Sensors : Obstacle detector")

# GRID ENVIRONMENT
grid = [
    [0, 0, 0, 0, 0],
    [1, 1, 0, 1, 0],
    [0, 0, 0, 1, 0],
    [0, 1, 0, 0, 0],
    [0, 0, 0, 1, 0]
]
ROWS = len(grid)
COLS = len(grid[0])
START = (0, 0)
GOAL = (4, 4)
# DISPLAY GRID
def display_grid(path=None):
    temp = [row[:] for row in grid]
    if path:
        for x, y in path:
            if (x, y) != START and (x, y) != GOAL:
                temp[x][y] = "*"
    temp[START[0]][START[1]] = "S"
    temp[GOAL[0]][GOAL[1]] = "G"
    print("\nGrid Environment:\n")
    for row in temp:
        print(row)

```

```

# VALID MOVE

```

```

def is_valid(x, y):
    return (
        0 <= x < ROWS and
        0 <= y < COLS and
        grid[x][y] != 1
    )

```

```

# NEIGHBOR FUNCTION

```

```

def get_neighbors(node):
    x, y = node
    moves = [
        (-1, 0),
        (1, 0),
        (0, -1),
        (0, 1)
    ]
    neighbors = []
    for dx, dy in moves:
        nx = x + dx
        ny = y + dy

        if is_valid(nx, ny):
            neighbors.append((nx, ny))

    return neighbors

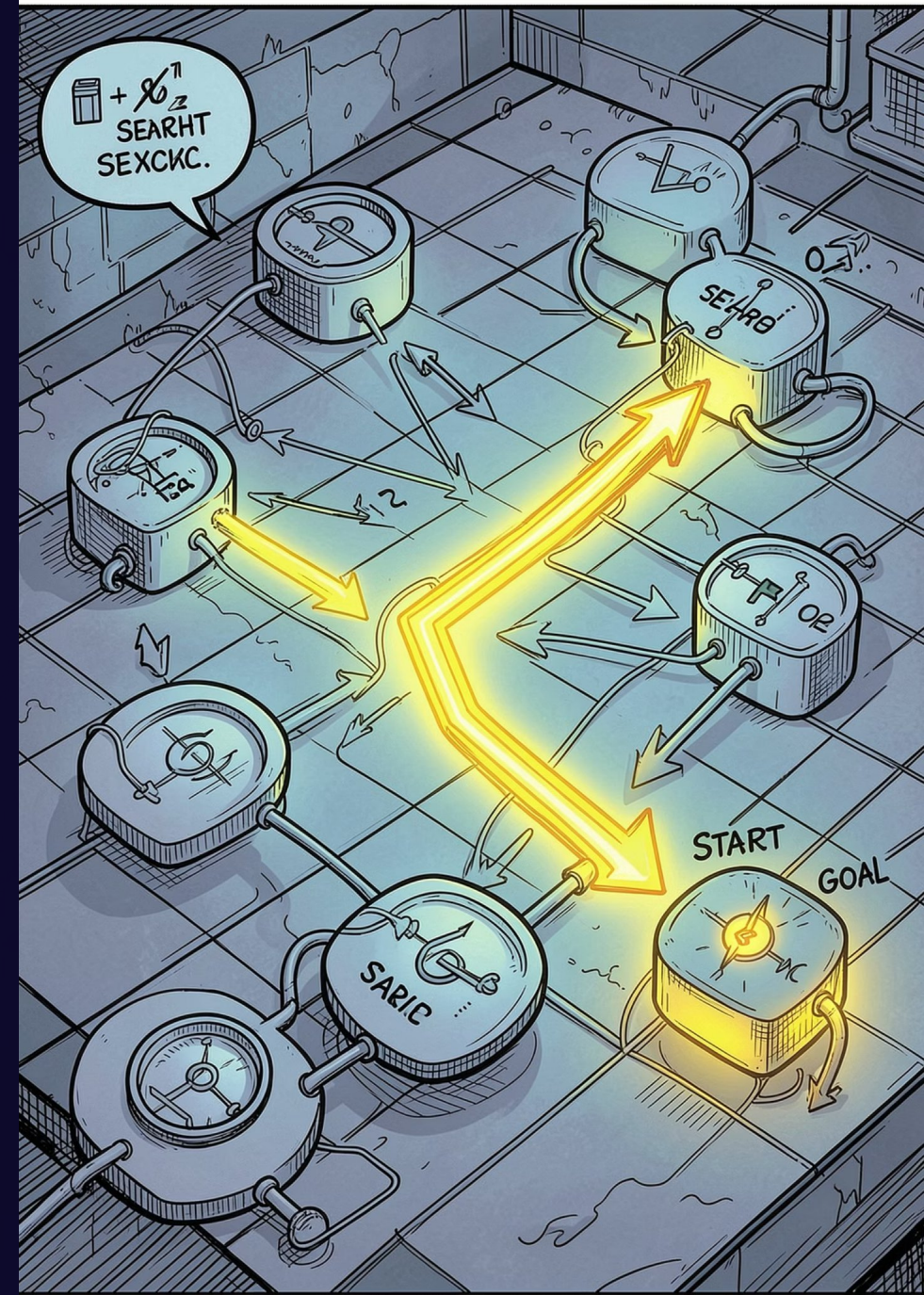
```



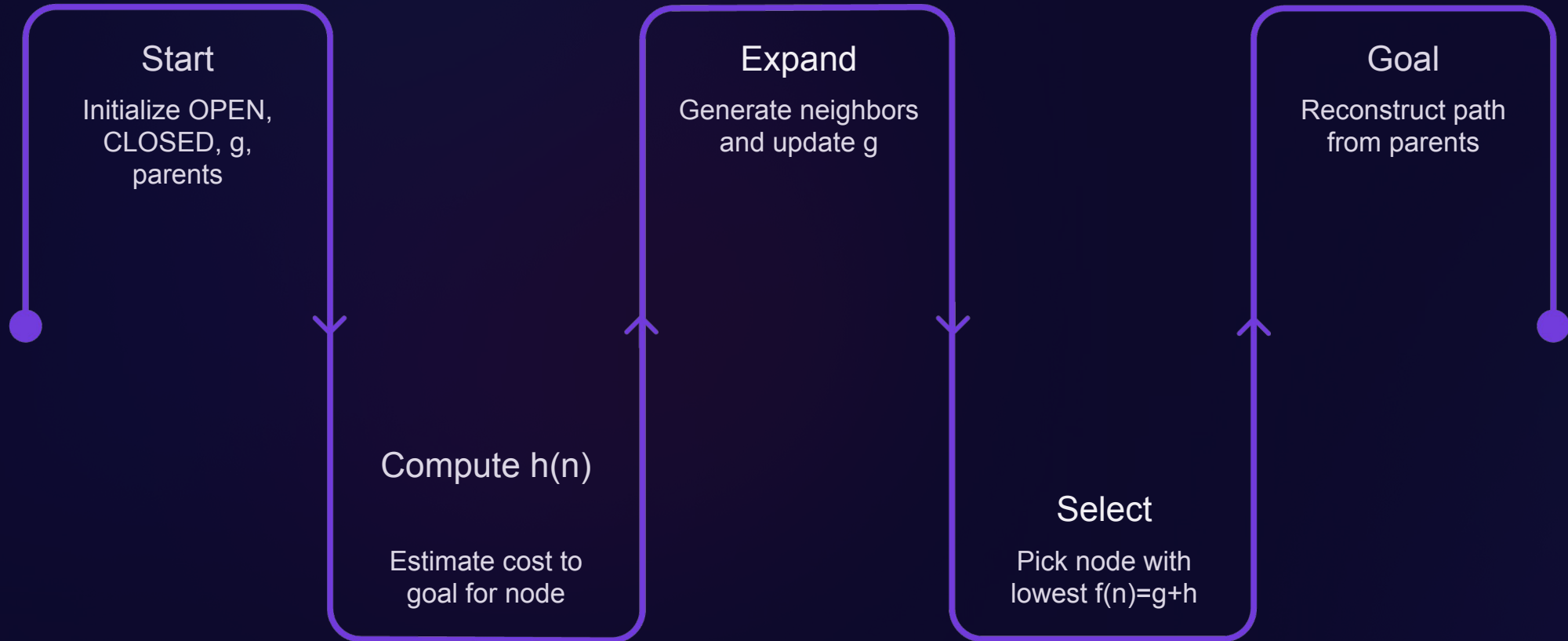
# CO2 — A\* Search: Core Concept

A\* is an informed graph search that combines cost-so-far  $g(n)$  with heuristic estimate  $h(n)$ :  $f(n)=g(n)+h(n)$ . It returns an optimal shortest path when  $h(n)$  is admissible (never overestimates true cost).

- Uses an OPEN frontier (priority queue by  $f$ )
- Expands nodes with lowest  $f$  first to balance exploration and goal-directedness
- Handles obstacles by treating blocked cells as impassable



# CO2 — A\* Working (Step-by-step)



Caption: Follow this sequence during planning — compute g/h, expand neighbors, maintain parent pointers, and reconstruct the solution once the goal node is popped from OPEN.



```
# ----- CO2 -----
```

```
# A* SEARCH ALGORITHM
```

```
def heuristic(a, b):
```

```
    return abs(a[0] - b[0]) + abs(a[1] - b[1])
```

```
def astar():
```

```
    pq = [(0, 0, START, [])]
```

```
    visited = set()
```

```
    while pq:
```

```
        f, g, node, path = heapq.heappop(pq)
```

```
        if node in visited:
```

```
            continue
```

```
        visited.add(node)
```

```
        path = path + [node]
```

```
        if node == GOAL:
```

```
            return path
```

```
        for neighbor in get_neighbors(node):
```

```
            new_g = g + 1
```

```
            h = heuristic(neighbor, GOAL)
```

```
            new_f = new_g + h
```

```
            heapq.heappush(
```

```
                pq,
```

```
                (new_f, new_g, neighbor, path)
```

```
            )
```

```
    return None
```



# CO3 — Constraint Satisfaction (CSP) in Path Planning



Treat path generation as a CSP with constraints enforced at expansion time and during successor generation.

01

**Constraint: No repeated nodes**

Detect cycles by tracking visited states in the path; prune revisits.

02

**Constraint: No obstacle traversal**

Validate each successor against the occupancy grid before insertion to OPEN.

03

**Constraint: Valid path only**

Ensure connectivity (reachable goal) and respect energy/charging limits when required.

```
# ----- CO3 -----
```

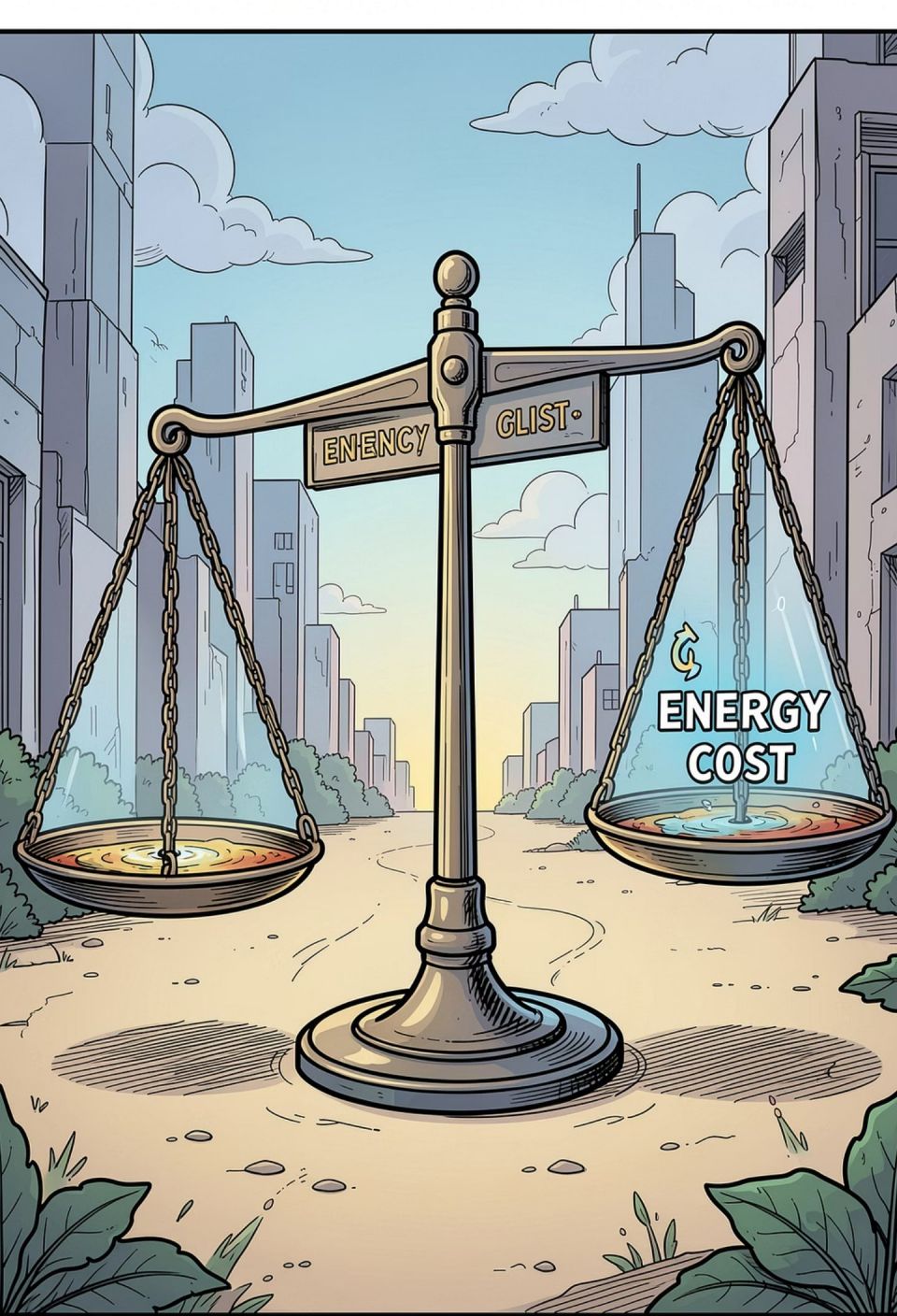
```
# CSP CONSTRAINT CHECKING
```

```
def constraint_check(path):
```

```
    if len(path) != len(set(path)):  
        return False
```

```
    for x, y in path:  
        if grid[x][y] == 1:  
            return False
```

```
    return True
```



# CO4 — Utility-Based Agent

Define a utility function to compare candidate plans and make trade-offs when multiple feasible paths exist.

## Utility Formula

$$\text{Utility} = 100 - \text{PathLength} - \text{EnergyCost}$$

## Decision Rule

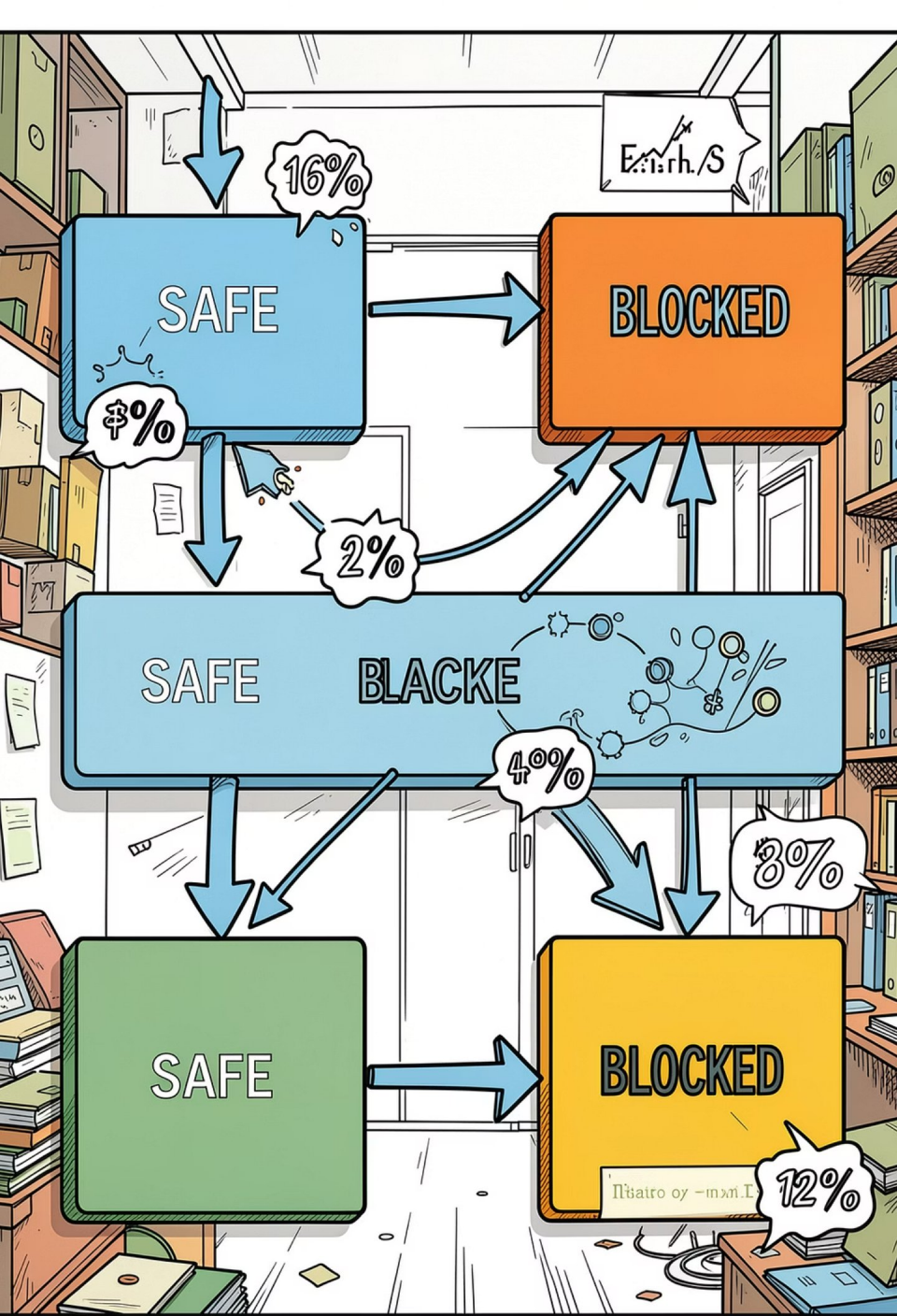
Choose plan with highest utility;  
tie-break on lower runtime or  
higher safety margin.

## Extensions

Incorporate battery thresholds, user preferences (quiet mode), or cleaning priority zones.



```
# ----- CO4 -----  
  
# UTILITY FUNCTION  
  
def utility_function(path):  
  
    path_length = len(path)  
  
    energy_cost = path_length * 0.5  
  
    utility = 100 - path_length - energy_cost  
  
    return utility
```



# CO5 — Markov Model for Uncertainty

Model local cell status as a 2-state Markov chain: {SAFE, BLOCKED}. Use transition probabilities to predict short-term changes and incorporate into planning (risk-aware A\* variants).



## SAFE

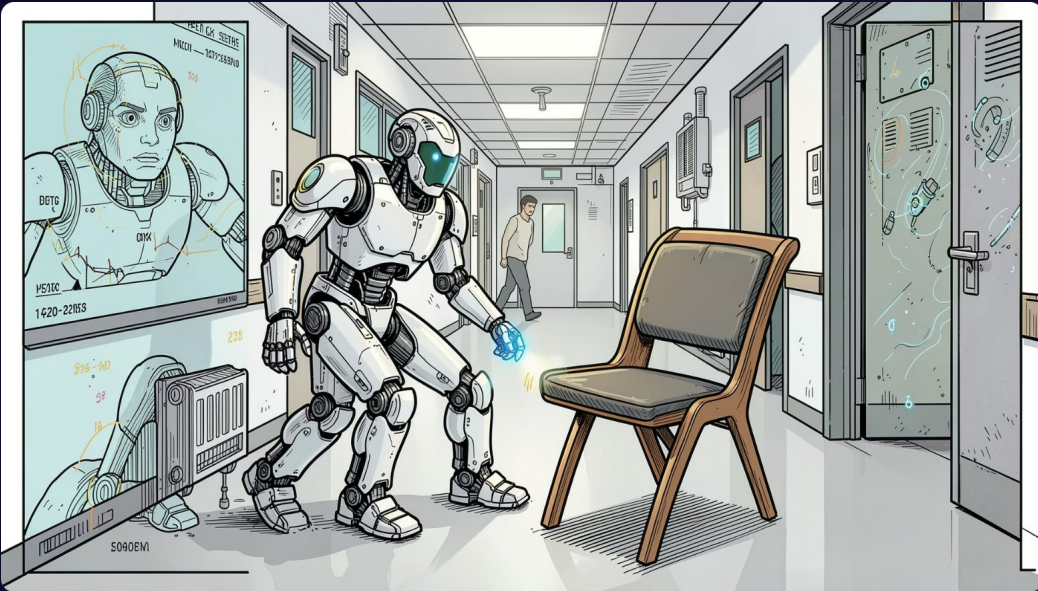
Cell expected traversable; low immediate collision risk.



## BLOCKED

Cell likely occupied or unsafe; high traversal cost or disallowed.

# CO5 — Uncertainty Handling Strategies



- Use predicted transition probabilities to inflate heuristic:  
 $h'(n) = h(n) + \text{riskPenalty}$
- Plan replanning: run A\* with current best belief; replan when observations contradict belief
- Safe fallback behaviors: wait, re-route, or backtrack to charger if high uncertainty
- Combine with particle filtering or occupancy grid smoothing for better belief estimates

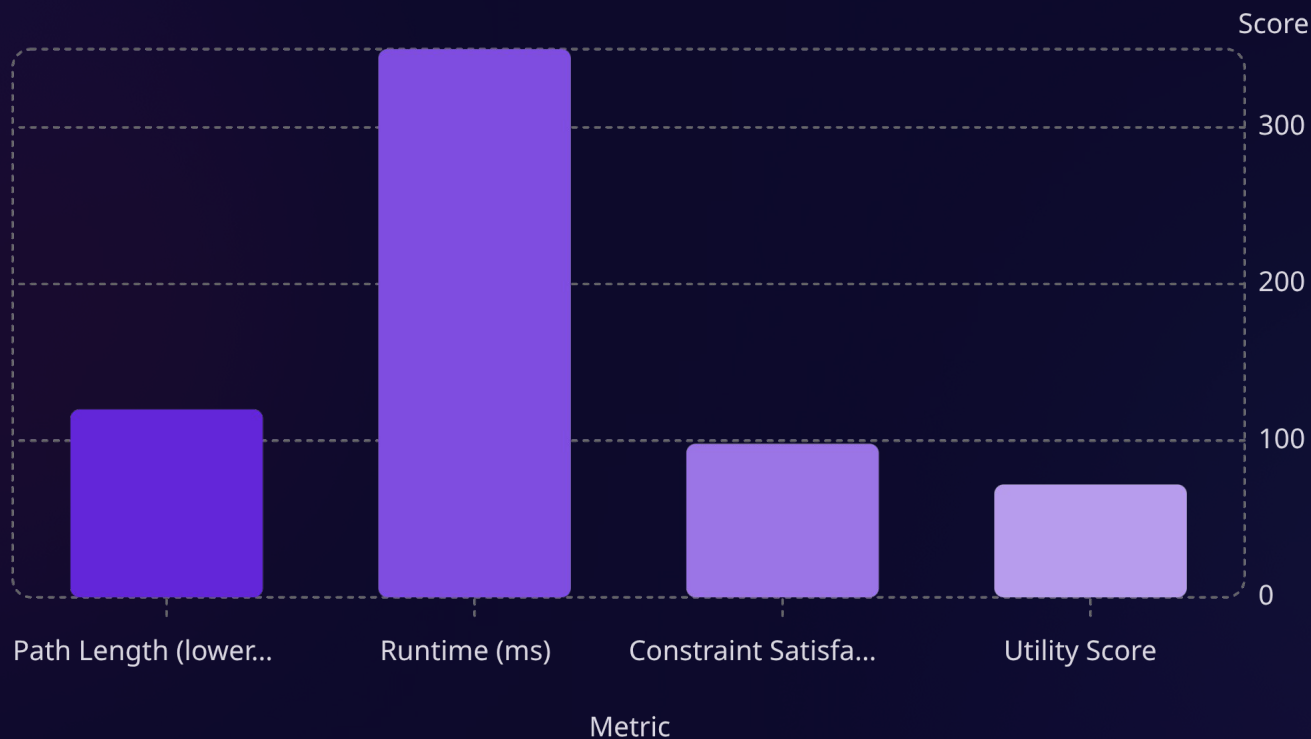
⚠ Keep replanning frequency bounded to avoid thrashing (e.g., minimum time between replans).



```
# ----- CO5 -----  
# MARKOV MODEL  
  
def markov_next_state(current_state):  
  
    transitions = {  
        "SAFE": ["SAFE", "BLOCKED"],  
        "BLOCKED": ["SAFE", "BLOCKED"]  
    }  
  
    return random.choice(transitions[current_state])
```

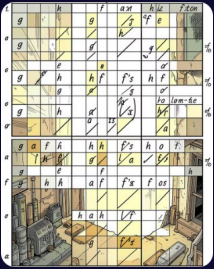


# CO6 — Performance Analysis Metrics



Use these metrics to compare algorithms (A\* variants, D\*, greedy). Report across multiple seeds/environments and include confidence intervals.

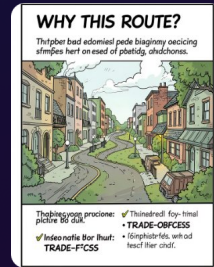
# CO6 — Explainable AI & Next Steps



## Decision Trace

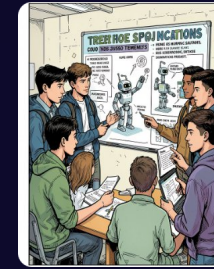
Log parent,  $g(n)$ ,  $h(n)$ ,  $f(n)$   
for each expanded node.

Present as human-  
readable sequence for  
debugging and grading.



## Human Explanation

Translate metrics into  
plain-language rationales:  
energy vs. coverage trade-  
offs, safety reasons for  
rerouting.



## Course Deliverables

Provide code (A\*/utility  
agent/CSP checks),  
datasets/environments,  
evaluation scripts, and  
README with  
reproduction steps.

Next steps: integrate online localization improvements, test with dynamic obstacles, and prepare a short demo script that highlights explainability.



```
# ----- CO6 -----
```

## # PERFORMANCE ANALYSIS

```
start_time = time.time()
path = astar()
end_time = time.time()
runtime = end_time - start_time
```

## # OUTPUTS

```
print("\nPath Found:\n")
print(path)
print("\nPath Length :", len(path))
print("\nExecution Time :", runtime)
valid = constraint_check(path)
print("\nConstraint Satisfaction :", valid)
utility = utility_function(path)
print("\nUtility Score :", utility)
display_grid(path)
```

## # UNCERTAINTY HANDLING

```
print("\n UNCERTAINTY HANDLING")
```

```
state = "SAFE"
```

```
for i in range(5):
```

```
    next_state = markov_next_state(state)
```

```
    print("Current State :", state,
          "--> Next State :", next_state)
```

```
    state = next_state
```

```
print("\n EXPLAINABLE AI")
print("1. Grid environment created.")
print("2. Path planning completed.")
print("3. Obstacles avoided successfully.")
print("4. Utility evaluated for efficiency.")
print("5. Uncertainty handled using Markov model.")
print("6. Optimized cleaning path generated.")
```

THANK YOU